

DEV-591: Document Redis persistence configuration (RDB, AOF)

Department: DevOps

Prepared by: Thanusha Bai V

Date: 17-sept-2026

Task: DEV-587-Redis Security Hardening

1. Executive Summary

This report documents Redis persistence mechanisms – RDB (snapshots) and AOF (Append-Only File) – and validates their behavior on a reference Redis 8.10.1 instance. Both mechanisms were tested live: an RDB snapshot was triggered and verified on disk, AOF was enabled and observed across a restart, and a critical pitfall was discovered (CONFIG SET is runtime-only and resets on restart). The findings establish Glynac's recommended persistence configuration per data class: RDB for caches, AOF (with everysec) for sessions and queues, and hybrid mode for critical workloads.

2. What RDB and AOF Are

2.1 RDB (Redis Database Snapshot)

RDB writes a **point-in-time binary snapshot** of the entire dataset to `dump.rdb`. It's fast to load but you only recover data as of the **last snapshot** – anything written after is lost.

- **Triggered by:** save thresholds in config, manual BGSAVE/SAVE, SHUTDOWN, or replication.
- **Pros:** compact file, fast restart, low runtime overhead.
- **Cons:** data loss window = time since last snapshot.

2.2 AOF (Append-Only File)

AOF logs **every write command** to disk as it happens. On restart, Redis replays the log to rebuild the dataset.

- **Triggered by:** every write when appendonly yes.
- **appendfsync options:**
 - `always` – fsync every write (safest, slowest)
 - `everysec` – fsync once per second (balanced – default)
 - `no` – let the OS decide (fastest, riskiest)
- **Pros:** minimal data loss window (~1s with everysec).
- **Cons:** bigger files, slower restarts, periodic rewrites needed.

2.3 Redis 7+ Multi-Part AOF

Redis 7 replaced the single AOF file with a directory of three files:

File	Purpose
appendonly.aof.N.base.rdb	Base snapshot (RDB format)
appendonly.aof.N.incr.aof	Incremental append log
appendonly.aof.manifest	Index tying base + incr together

This makes rewrites cheaper and recovery faster.

3. Reference Configuration (Baseline)

From CONFIG GET on the reference instance:

Setting	Value	Meaning
save	3600 1 300 100 60 10000	Snapshot after 1 change/1h, 100 changes/5m, 10000 changes/1m
appendonly	no	AOF disabled at baseline
appendfsync	everysec	(Only applies when AOF is on)
dir	/var/lib/redis	Data directory
dbfilename	dump.rdb	RDB filename

Baseline verdict: RDB-only. Data loss window equals time since last snapshot. Acceptable for pure caches, risky for sessions/queues.

4. RDB Test Results

4.1 Steps

1. Wrote 2 test keys (rdb:test, rdb:test2).

2. Triggered BGSAVE.
3. Verified status via INFO persistence.
4. Confirmed file on disk.

4.2 Output

```
rdb_changes_since_last_save:0
rdb_bgsave_in_progress:0
rdb_last_save_time:1789659308
rdb_last_bgsave_status:ok
rdb_last_bgsave_time_sec:0
rdb_saves:1
rdb_last_cow_size:614400
```

```
-rw-rw---- 1 redis redis 188 Sep 17 21:05 dump.rdb
```

4.3 Interpretation

- Snapshot completed in **0 seconds**, status: ok.
- `rdb_changes_since_last_save:0` → all pending writes are now in the file.
- `dump.rdb` is 188 bytes, owned by `redis:redis`, mode 660.
- **Durability gap confirmed:** any write *after* `rdb_last_save_time` will be lost on crash until the next snapshot.

5. AOF Test Results

5.1 Enabling AOF

```
CONFIG SET appendonly yes
```

Result: OK. After ~3 seconds, AOF files appeared:

```
appendonlydir/
|—— appendonly.aof.1.base.rdb      188 B
|—— appendonly.aof.1.incr.aof      88 B
|—— appendonly.aof.manifest        102 B
```

5.2 Status After Enable

```
aof_enabled:1
aof_last_bgrewrite_status:ok
aof_last_write_status:ok
aof_current_size:276
aof_base_size:188
aof_rewrites:1
```

- aof_rewrites:1 → Redis performed an initial BGREWRITEAOF to create the base file.
- All statuses ok.

5.3 Post-Enable Write

```
SET aof:test "value-written-after-aof-enabled"
```

Written **after** AOF was enabled – captured in incr.aof.

5.4 Restart Test

All 3 keys survived:

```
rdb:test → "value-before-save"
rdb:test2 → "value2"
aof:test → "value-written-after-aof-enabled"
```

But aof_enabled:0 after restart. ⚠

6. ⚠ Critical Finding — CONFIG SET is Runtime-Only

CONFIG SET appendonly yes changes the **running instance only**. It is **not** written to redis.conf. On restart, Redis re-reads the config file (which still says appendonly no) and disables AOF.

Why the data still survived: During the initial BGREWRITEAOF, Redis created appendonly.aof.1.base.rdb – a full snapshot at the moment AOF was enabled. That base file updated dump.rdb on disk. On restart, Redis loaded from the RDB file (rdb_last_load_keys_loaded:5), which contained all 5 keys including aof:test.

This is a trap. Teams often think they've enabled AOF permanently when they've only enabled it for the current process lifetime.

6.1 Fix – Persist the change

```
CONFIG SET appendonly yes
```

```
CONFIG REWRITE
```

CONFIG REWRITE writes the runtime config back into redis.conf.
Verified:

```
$ sudo grep -E "^appendonly|^appendfsync" /etc/redis/redis.conf
```

```
appendonly yes
```

```
appendfsync everysec
```

After systemctl restart redis-server:

```
CONFIG GET appendonly → "yes"
```

```
GET aof:test → "value-written-after-aof-enabled"
```

AOF now persists across restarts. 

7. RDB vs AOF — Decision Matrix for Glynac

Data class	Recommended mode	Why
Pure cache (page cache, query cache)	RDB only	Data is reconstructable; loss window acceptable
Sessions	AOF everysec	Losing active sessions is user-visible
Job queues	AOF everysec	Losing queued jobs breaks processing
Rate-limit counters	RDB or AOF everysec	Small loss acceptable
Critical state (billing, ledgers)	AOF always + RDB for backup	Zero-loss tolerance
Replicas	Match primary	Replicas follow the primary's persistence

Default recommendation: appendonly yes with appendfsync everysec for Glynac's production instances. Add RDB snapshots on top (save 900 1 300 100) for backup convenience.

8. Recommended Production Config

```
# RDB – periodic snapshots for backup
save 900 1
save 300 100
save 60 10000

# AOF – durability between snapshots
appendonly yes
appendfsync everysec

# Prevent AOF rewrites during heavy save load
no-appendfsync-on-rewrite no

# Auto-rewrite when AOF grows 100% and is at least 64MB
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb

# Truncated tail handling
aof-load-truncated yes

# Hybrid AOF (RDB preamble) – faster restart
aof-use-rdb-preamble yes
```

Do not forget:

```
CONFIG REWRITE      # after any runtime change
```

9. Operational Rules

1. **Any CONFIG SET change must be followed by CONFIG REWRITE** – or it dies on the next restart.
2. **Never disable RDB entirely** even with AOF on – RDB is your cheapest backup format.
3. **Test restores**, not just backups. A config that has never been restored from is not proven.
4. **Watch rdb_last_bgsave_status and aof_last_bgrewrite_status** in monitoring (see DEV-594).
5. **Monitor rdb_changes_since_last_save** – a growing value means snapshots aren't keeping up.
6. **For disks with slow fsync**, consider appendfsync everysec over always and tune no-appendfsync-on-rewrite.

10. Summary

Area	Finding
Baseline persistence	RDB only
RDB save test	✅ BGSAVE succeeded, dump.rdb written (188 B)
AOF enable test	✅ 3-file multi-part AOF created, statuses ok
Restart test	✅ All keys survived – but AOF reset to off
Root cause	CONFIG SET is runtime-only
Fix	CONFIG REWRITE after CONFIG SET – verified persistent
Recommended mode	AOF everysec + RDB snapshots for production

11. Recommendations for Glynac

1. Set appendonly yes + appendfsync everysec in redis.conf on all production instances.
2. Keep RDB enabled (save 900 1 300 100) for backup convenience.
3. Always run CONFIG REWRITE after runtime changes.
4. Use aof-use-rdb-preamble yes for faster restarts.
5. Include persistence status in monitoring (feeds into DEV-594).
6. Test restores against a scratch instance before trusting backups (see DEV-593).

This report validates RDB and AOF behavior against a live Redis 8.10.1 instance. All test outputs are reproduced verbatim from the reference lab.

END OF REPORT